

CourtBook

Documentación técnica del proyecto — archivo por archivo

App web de reserva de canchas deportivas

Proyecto Final · Interfaces y Programación Web

Eva · Junio 2026

Stack: React 19 · Vite · Firebase (Auth + Firestore) · Tailwind CSS 4

1. Visión general

CourtBook es una aplicación web de reserva de canchas deportivas. Conecta a **clientes** (jugadores que buscan dónde jugar) con **dueños** de complejos que publican y gestionan sus canchas. Todo el estado vive en Firebase: la autenticación en Firebase Auth y los datos en Firestore, una base NoSQL en tiempo real. El frontend es una SPA en React; no hay servidor propio: la seguridad y la lógica de acceso se resuelven con las **reglas de Firestore**.

Este documento explica qué hace cada archivo **escrito a mano** del proyecto (se omiten los autogenerados por Vite/Firebase y las dependencias), cómo se relacionan entre sí, y detalla las reglas de seguridad de Firestore.

Idea mental rápida: `main.jsx` monta la app dentro del `AuthProvider` → `App.jsx` decide qué página mostrar según sesión y rol → cada página lee/escribe en Firestore en tiempo real con `onSnapshot` → las reglas de `firestore.rules` validan cada operación del lado del servidor.

Mapa de carpetas

Ruta	Qué contiene
<code>src/main.jsx</code>	Punto de entrada: monta React y el proveedor de auth
<code>src/App.jsx</code>	Router y guardas de ruta por sesión/rol
<code>src/context/</code>	Estado global de autenticación (<code>AuthContext</code>)
<code>src/firebase/</code>	Inicialización y configuración de Firebase
<code>src/hooks/</code>	Hook reutilizable anti doble-submit
<code>src/components/</code>	Navbar compartida (perfil + notificaciones)
<code>src/pages/</code>	Las pantallas: Login, Home, Reserve, MyReservations, OwnerPanel, NotFound
<code>firestore.rules</code>	Reglas de seguridad de la base de datos
<code>.github/workflows/</code>	CI/CD: deploy automático a Firebase Hosting

2. Arranque y configuración

`src/main.jsx` núcleo

Punto de entrada de la aplicación. Crea la raíz de React con `createRoot` y renderiza `<App/>` envuelto en dos cosas: `<StrictMode>` (chequeos extra de React en desarrollo) y `<AuthProvider>`, que pone el estado de sesión a disposición de toda la app.

Se relaciona con: `AuthContext.jsx` (lo envuelve) y `App.jsx` (lo renderiza). Es la primera pieza que se ejecuta.

src/firebase/config.js núcleo

Inicializa Firebase y exporta las herramientas que usa el resto del proyecto: `auth` (autenticación), `db` (Firestore) y `analytics`. Las credenciales se leen de variables de entorno `VITE_FIREBASE_*` (nunca van hardcodeadas), lo que permite usar distintos proyectos para desarrollo y producción.

Detalle clave: en vez de un Firestore común, inicializa con `persistentLocalCache` + `persistentMultipleTabManager`. Eso activa el **cache offline en IndexedDB** desde el arranque y permite que varias pestañas compartan ese cache sin conflictos. La app sigue mostrando datos aunque la conexión falle.

Se relaciona con: lo importan absolutamente todas las páginas y componentes que tocan datos o sesión (`auth` y `db`).

src/context/AuthContext.jsx núcleo

El corazón de la autenticación. Crea un contexto global de React que expone `user`, `userRole`, `loading` y `setUser` a toda la app mediante el hook `useAuth()`.

Se suscribe a `onAuthStateChanged` de Firebase, que se dispara cada vez que cambia el estado de sesión. Cuando hay un usuario logueado, busca su documento en `users/{uid}` de Firestore:

- Si existe, fusiona los datos de Firebase Auth (uid, email, foto) con los de Firestore (rol, whatsapp, nombre) en un solo objeto `user`.
- Si no existe (usuario nuevo), espera 500ms y hace `reload()` para capturar el `displayName` que el registro acaba de escribir, y recién ahí crea el documento con `role: "client"`.

Bloquea el render de la app (`{!loading && children}`) hasta resolver la sesión, para evitar parpadeos de rutas incorrectas o redirecciones prematuras.

Se relaciona con: lo consumen `App.jsx` (guardas), `Navbar.jsx`, y todas las páginas que necesitan saber quién es el usuario.

src/App.jsx núcleo

Define el ruteo con React Router y, sobre todo, las **cuatro guardas de acceso** que protegen cada página:

Guarda	Regla
<code>PublicRoute</code>	Solo si NO hay sesión (ej. el Login). Si ya está logueado, redirige a <code>/</code> .
<code>PrivateRoute</code>	Requiere sesión. Sin login → redirige a <code>/login</code> .
<code>OwnerRoute</code>	Requiere rol <code>owner</code> o <code>superadmin</code> . Un cliente queda fuera.
<code>RootRoute</code>	En la raíz redirige según rol: <code>owner/superadmin</code> → <code>/owner</code> , <code>cliente</code> → <code>Home</code> .

Se relaciona con: usa `useAuth()` de `AuthContext` para decidir, e importa todas las páginas de `src/pages`.

Tabla de rutas

Ruta	Componente	Acceso
------	------------	--------

<code>/login</code>	Login.jsx	Público (sin sesión)
<code>/</code>	Home.jsx	Cliente (owner se redirige)
<code>/owner</code>	OwnerPanel.jsx	Owner / Superadmin
<code>/reserve/:courtId</code>	Reserve.jsx	Con sesión
<code>/my-reservations</code>	MyReservations.jsx	Con sesión
<code>*</code>	NotFound.jsx	Todos

3. Utilidades compartidas

src/hooks/useAsyncAction.js hook

Hook reutilizable que protege cualquier acción asíncrona contra el **doble-click / multi-submit**. Combina dos mecanismos: un `useRef ( inFlight )` que bloquea de forma sincrónica e inmediata —antes de cualquier re-render— y un `useState ( loading )` que da feedback visual al botón. Expone `run(fn)`, `loading` y `error`.

Si una acción ya está en curso, el segundo click se ignora. Esto evita reservas o escrituras duplicadas en Firestore.

Se relaciona con: lo usan **Reserve.jsx** (al reservar), **OwnerPanel.jsx** (agregar/editar/eliminar canchas, confirmar/rechazar) y **MyReservations.jsx** (cancelar). Cada botón crítico usa `disabled={loading}`.

src/components/Navbar.jsx componente

Barra de navegación compartida por las páginas internas. Es bastante más que un menú: incluye el **avatar con menú de perfil** (con fallback a la inicial si la foto falla), el cierre de sesión, y —para owners— dos cosas importantes:

- **Campana de notificaciones en tiempo real:** un `onSnapshot` escucha las reservas `pending` del owner. El badge con el contador y el dropdown se actualizan solos, sin recargar. Desde ahí el owner puede confirmar o rechazar, lo que también sincroniza el `slot` correspondiente.
- **Edición del WhatsApp de contacto:** el owner guarda su número, que se escribe en su doc `users` y se espeja en `ownerContacts` (dato público acotado que luego lee Reserve para armar el chat).

Incluye además detección de clicks fuera de los menús para cerrarlos, y limpieza de listeners al desmontar para evitar fugas de memoria.

Se relaciona con: usa `useAuth()`, escribe en `users / ownerContacts / reservations / slots`. La renderizan Home, OwnerPanel, Reserve y MyReservations.

4. Páginas

src/pages/Login.jsx página · pública

Pantalla de acceso con **tres modos** en un solo componente, controlados por el estado `mode` : iniciar sesión, registrarse y recuperar contraseña. Soporta login con Google (popup) y con email/contraseña.

Tiene una función `errorMessage()` que traduce los códigos de error de Firebase a mensajes claros en español, agrupando casos como "usuario no encontrado" y "contraseña incorrecta" en un único mensaje genérico para no revelar si un email existe. En el registro, tras crear el usuario llama a `updateProfile` con el nombre; ese `displayName` es justamente lo que **AuthContext** espera para crear el documento en Firestore.

Se relaciona con: `auth` de Firebase; al loguearse, **AuthContext** reacciona y **App.jsx** redirige según el rol.

src/pages/Home.jsx página · cliente

Catálogo de canchas que ve el cliente. Trae de Firestore las canchas con `available == true` y las muestra como tarjetas (emoji por deporte, nombre, ubicación, precio por hora). Cada tarjeta tiene un botón "Reservar" que lleva a `/reserve/:courtId`.

Optimización propia: usa `sessionStorage` como cache. Muestra al instante las canchas cacheadas y solo re-renderiza si Firestore devuelve datos distintos, evitando parpadeos. No muestra el botón reservar en la cancha propia del usuario.

Se relaciona con: lee `courts` ; navega a **Reserve.jsx**; usa **Navbar**.

src/pages/Reserve.jsx página · cliente

El flujo de reserva. Lee el `courtId` de la URL, carga la cancha y pre-carga el WhatsApp del dueño desde `ownerContacts` . El cliente elige una fecha y ve una grilla de horarios (9 a 20hs) cuyo estado se actualiza **en tiempo real** con `onSnapshot` sobre la colección `slots` : libre, pendiente o confirmado.

Al reservar pasan dos cosas: (1) se abre WhatsApp con un mensaje pre-armado al dueño — `window.open` se llama *antes* de cualquier `await` para que los bloqueadores de popups no lo cancelen— y (2) se ejecuta una **transacción atómica** (`runTransaction`) que crea el `slot` y la `reservation` juntos. El `slot` tiene un ID determinista (`courtId_fecha_hora`): si dos clientes reservan el mismo turno a la vez, la segunda transacción detecta el slot ocupado y falla. Ese es el **candado anti doble-reserva**.

Se relaciona con: lee `courts / ownerContacts` , escribe `slots / reservations` ; usa `useAsyncAction` y **Navbar**.

src/pages/MyReservations.jsx página · cliente

Historial de reservas del cliente. Un `onSnapshot` escucha sus reservas: cuando el owner confirma o rechaza, la UI cambia sola sin recargar. Cada reserva muestra un badge de color según su estado (pendiente / confirmada / cancelada) y, si sigue `pending`, un botón para cancelarla (que también libera el `slot` marcándolo `cancelled`).

Los nombres de las canchas se resuelven aparte y se guardan en un mapa local para no re-consultar Firestore por cada reserva.

Se relaciona con: lee `reservations / courts`, actualiza `reservations / slots`; usa `useAsyncAction` y **Navbar**.

src/pages/OwnerPanel.jsx página · owner

El panel del dueño y el archivo más grande del proyecto. Concentra toda la gestión:

- **CRUD de canchas:** agregar (hasta 6), editar y eliminar, con validación de campos. Un `onSnapshot` mantiene la grilla de canchas siempre al día.
- **Agenda del día:** una grilla por cancha y horario que muestra los turnos pendientes y confirmados de una fecha navegable, con métricas al pie (turnos, total facturado, % de ocupación).
- **Modales:** uno para ver el calendario de horarios de una cancha (listener propio que se activa al seleccionarla), otro para el detalle de cada reserva con botones de confirmar/rechazar.

Usa una instancia de `useAsyncAction` por cada acción crítica para que los loadings sean independientes. Confirmar o rechazar aplica **actualizaciones optimistas** y sincroniza el `slot`. Para evitar índices compuestos en Firestore, ciertos filtros de fecha se hacen del lado del cliente.

Se relaciona con: lee/escibe `courts`, `reservations` y `slots`; usa `useAuth`, `useAsyncAction` y **Navbar**. Es la contraparte del owner frente al flujo del cliente en Reserve/MyReservations.

src/pages/NotFound.jsx página · 404

Página simple para rutas inexistentes (`*` en el router). Muestra un 404 con un botón para volver al inicio.

5. Modelo de datos (Firestore)

Firestore organiza la información en colecciones de documentos. CourtBook usa cinco:

Colección	Campos principales	Para qué
<code>users</code>	name, email, role, whatsapp, photoURL	Perfil y rol de cada usuario (ID = uid). Privado a su dueño.
<code>ownerContacts</code>	whatsapp	Contacto público acotado del owner, sin datos sensibles. Lo lee Reserve.
<code>courts</code>	name, sport, price, location, available, ownerId	Las canchas publicadas por los owners.
<code>reservations</code>		Las reservas. <code>status</code> : pending → confirmed cancelled.

courtId, clientId, clientName,
ownerId, date, startTime, endTime,
status, createdAt

slots

courtId, date, startTime, status

Disponibilidad sin datos personales. ID determinista

`courtId_date_startTime` : es el candado anti doble-reserva.

Por qué existen `slots` y `ownerContacts` por separado: `slots` permite bloquear un turno de forma atómica sin exponer quién reservó; `ownerContacts` deja que cualquier cliente lea solo el WhatsApp del dueño sin poder ver el resto de su perfil privado (email, etc.).

6. Reglas de seguridad de Firestore

El archivo `firestore.rules` es la verdadera capa de seguridad: como no hay backend propio, define quién puede leer y escribir cada documento. Las reglas se evalúan en los servidores de Firebase, así que no se pueden saltar desde el cliente. Usa `rules_version = '2'`.

`users/{userId}` — perfil privado

- **read / create:** solo el propio usuario sobre su propio documento (`request.auth.uid == userId`). Antes la lectura estaba abierta a cualquier autenticado y filtraba email/whatsapp/nombre/rol de todos; se cerró.
- **update:** el usuario puede editar su doc **pero nunca el campo `role`** — se valida que `role` no esté entre las claves modificadas. Así nadie se auto-asciende a owner.

`ownerContacts/{ownerId}` — contacto público

- **read:** cualquier usuario autenticado (Reserve lo necesita para armar el chat de WhatsApp).
- **write:** solo el propio owner, y solo el campo `whatsapp` (se valida con `hasOnly(['whatsapp'])` y que sea string). Nada más puede escribirse acá.

`courts/{courtId}` — canchas

- **read:** cualquier autenticado (el catálogo es público para usuarios logueados).
- **create:** solo si el rol del usuario (leído de su doc `users`) es `owner` o `superadmin`, el `ownerId` coincide con quien crea, y el esquema valida: nombre no vacío y ≤ 80 caracteres, precio numérico ≥ 0 , ubicación ≤ 120 caracteres.
- **update / delete:** solo el dueño de la cancha, y en update no puede cambiar el `ownerId` (no puede "regalar" ni robar canchas).

slots/{slotId} — disponibilidad (anti doble-reserva)

- **read:** cualquier autenticado.
- **create:** solo con `status == 'pending'` y exactamente las claves `courtId`, `date`, `startTime`, `status`. Como el ID es determinista (`courtId_date_startTime`), el "create solo si no existe" es el candado server-side: la segunda escritura concurrente del mismo turno falla.
- **update:** solo puede cambiar el campo `status`, y solo a `confirmed` / `cancelled`, o reabrir un turno cancelado (`cancelled` → `pending`) para que vuelva a reservarse.

reservations/{resId} — reservas

- **read:** solo el cliente dueño de la reserva o el owner de la cancha (antes estaba abierta y filtraba nombres/ patrones de clientes).
- **create:** el `clientId` debe ser quien crea, no puede reservarse a sí mismo (`clientId != ownerId`), el `status` se fuerza a `pending` (nadie se auto-confirma) y se valida el esquema completo de campos.
- **update:** solo cambia `status`. El **owner** puede confirmar o cancelar reservas de su cancha; el **cliente** solo puede cancelar su propia reserva mientras está pendiente.

7. Build, configuración y despliegue

vite.config.js config

Configuración de Vite (el bundler/servidor de desarrollo). Registra dos plugins: el de React y el de Tailwind CSS 4 (que en la v4 se integra como plugin de Vite en vez de PostCSS).

eslint.config.js config

Configuración de ESLint (linter). Aplica las reglas recomendadas de JS, los plugins de React Hooks y React Refresh, e ignora `dist`. Tiene una regla a medida que permite variables sin usar si empiezan con mayúscula o guion bajo.

firebase.json config

Le dice a Firebase dos cosas: dónde están las reglas de Firestore (`firestore.rules`) y cómo servir el hosting. Publica la carpeta `dist` (el build) y usa un `rewrite` de todas las rutas a `/index.html`, necesario para que el routing del lado del cliente de una SPA funcione al recargar.

index.html config

El HTML base de la SPA. Define el `<div id="root">` donde React monta todo y carga `src/main.jsx` como módulo. Título: "Reservá Tu Cancha".

src/index.css config

Una sola línea: `@import "tailwindcss";`. Importa todo Tailwind. Todo el estilado de la app se hace con clases de Tailwind directamente en el JSX.

.github/workflows/ CI/CD

Dos workflows de GitHub Actions (autogenerados por la CLI de Firebase pero parte del pipeline):

- **firebase-hosting-merge.yml**: al hacer push o merge a `main`, instala dependencias, hace el build (inyectando las variables `VITE_FIREBASE_*` como secrets) y despliega a producción (`channelId: live`).
- **firebase-hosting-pull-request.yml**: en cada Pull Request, genera un deploy de preview para revisar los cambios antes de mergear.

Existe además un comando propio **.claude/commands/deploy.md** que automatiza el pre-deploy: verifica el branch, chequea el `.env`, corre lint y build, pide confirmación y pusha a `main`.

Nota sobre archivos no incluidos: `README.md` y `src/App.css` son sobras de la plantilla por defecto de Vite (no reflejan el proyecto), y `package.json` / `package-lock.json` solo listan dependencias. No forman parte de la lógica escrita de CourtBook, por eso no se documentan en detalle.

8. Cómo se conecta todo (flujo completo)

Un recorrido de punta a punta que sirve para la presentación:

1. **Arranque:** `main.jsx` monta la app dentro de `AuthProvider`, que resuelve la sesión vía Firebase Auth y carga el rol desde `users`.
2. **Ruteo:** `App.jsx` usa ese rol en sus guardas para mandar al cliente a Home y al owner a su panel.
3. **Cliente reserva:** en Home elige una cancha → en Reserve elige fecha y horario → una transacción crea `slot` + `reservation (pending)` y se abre WhatsApp al dueño.
4. **Owner gestiona:** la campana del Navbar y el OwnerPanel escuchan en tiempo real las reservas `pending` y permiten confirmar o rechazar, lo que actualiza `reservation` y `slot`.
5. **Cliente ve el resultado:** MyReservations refleja el cambio al instante por `onSnapshot`.
6. **Seguridad transversal:** cada una de esas escrituras pasa por `firestore.rules`, que valida permisos y esquema del lado del servidor.
7. **Despliegue:** al mergear a `main`, GitHub Actions buildea y publica en Firebase Hosting automáticamente.